

Device Library API Developer Guide

When SlimCDDDevice library instantiates the device specific library API it returns an object of the type ISlimCDDDeviceAsync defined in SlimCDTypeLib. This interface provides an API for communicating with card readers and performing transactions. The instance also implements and can be cast to ICommManager, IInteractive, interfaces. Where ICommManager can be used to determine the library communication capabilities and IInteractive provides interactivity functionality that can be used when an input cannot be completed on the reader, but the POS app can provide it.

Using a Device Library

Here are the steps that demonstrate the library use:

1. Get the instance of the device specific API:

```
IAsemblyInfo selectedAssembly = ...;
IPlatformSpecific PlatformSpecific = ...;
CancellationToken cancellationToken = ...;
IResponse<ISlimCDDDeviceAsync> instanceResponse;

try
{
    instanceResponse = await Assemblies.GetInstance(
        selectedAssembly,
        PlatformSpecific,
        cancellationToken);
}
catch (OperationCanceledException)
{
    // The only exception that can be thrown by the call.
}

if (instanceResponse.IsFailure)
    error = instanceResponse.Message; // Contains the error message
else
    instance = instanceResponse.Data; // Contains the ISlimCDDDeviceAsync object
```

2. Connect to the device:

```
DeviceConnectRequest connectRequest = ...;
CancellationToken cancellationToken = ...;
IResponse<IDeviceInfo> connectResponse;

try
{
    connectResponse = await instance.DeviceConnectAsync(
        connectRequest,
        cancellationToken);
}
catch (OperationCanceledException)
{
    // The only exception that can be thrown by the call.
}

if (connectResponse.IsFailure)
    error = connectResponse.Message; // Contains the error message
else
    device = connectResponse.Data; // Contains the IDeviceInfo object
```

3. Initiate a transaction:

```
DeviceTransactionConfig config = ...;
DeviceTransactionRequest request = ...;
Func<DeviceTransactionReply, Task> replyCallback = ...;
Func<string, Task> statusCallback = ...;

await instance.DeviceProcessTransactionAsync(
    config,
    request,
    OnTransactionReplyReceived,
    OnStatusUpdate);

// The result will be returned in a call to OnTransactionReplyReceived
```

After the device successfully connected you can start calling the other methods that communicate with the device.

After you finished using the `ISlimCDDeviceAsync` instance call `instance.DeviceDisconnectAsync()`. This method will cause the library to disconnect from the device and it will also dispose the internal resources. After disconnecting there is no need to dispose the object, **however**, if `DeviceDisconnectAsync` was not called it required to call `Dispose` to avoid memory leaks.

ISlimCDDeviceAsync API

Most of the API methods is async and return `IResult` or `IResponse<T>` (Defined in `SlimCDTypeLib`). The result / response values indicate if the transaction was successful or failed and may contain the response data object. `IResponse<T>` is derived from `IResult`. The methods are expected not to throw exceptions unless specified or cancellation exceptions if the cancellation has been triggered.

Properties in `IResult` and `IResponse<T>`.

<code>IResult.IsSuccess</code> <code>IResult.IsFailure</code>	Indicates if the call to the method was successful or failed. The values are mutually exclusive, – if one is <code>True</code> – the other is <code>False</code> .
<code>Message</code>	In a case if the call to the method has failed this property will contain the error message. Otherwise this value will be <code>null</code> .
<code>IResponse<T>.Data</code>	In a case if the call to the method was successful it will contain the data object of the specified type.

Device Initialization

```
async Task<IResult> DeviceInitializeAsync(
    string statsUrl,
    string transUrl,
    DeviceDataLists dataLists,
    CancellationToken ctx,
    bool useDefaultProxy = false,
    IWebProxy proxy = null);
```

The call to this method is optional and only need to be done when you want to change one of the default values. It updates the URLs that is used to perform transactions and related calls to the SlimCD server, sets the logs callback and the Proxy data.

Parameters

statsUrl, transUrl	URLs to be used in the transaction process. The default URLs set in the library is set to: statsURL = "https://stats.slimcd.com"; transURL = "https://trans.slimcd.com"; Pass null in those arguments if you want to use the default values.
dataLists	Object that defines how to treat different card AIDs, categorizing them as Debit or Credit AIDs. Generally this shouldn't be changed unless instructed to do so. Pass null to use the default values.
ctx	Cancellation token for the operation. Not needed, unless an INI file is been used, otherwise this call is very fast.
useDefaultProxy	Boolean flag, if set to true then the Internet Explorer (IE) proxy settings set in Windows will be used for performing transactions.
proxy	Should contain the proxy object to be used to perform transactions. If useDefaultProxy is true then this object will be ignored. If specified as null – no proxy will be used.

Returns

IResult	This method does not throw exception, but rather will always return the result indicating a success or failure. In a case if a cancellation was requested the method will return a failure.
---------	---

Note. The API also contain a very similar method DeviceUpdateURLs that can be used if you just need to update the service URLs or set a proxy. It doesn't read INI files and it is faster. The additional debug argument is not been used and it is present there for a backward compatibility:

```
void DeviceUpdateURLs(
    bool debug,
    string statsURL,
    string transURL,
    bool UsePreconfigProxy = false,
    IWebProxy Proxy = null);
```

Connecting Device

```
async Task<IResponse<IDeviceInfo>> DeviceConnectAsync(
    DeviceConnectRequest connectionRequest,
    CancellationToken ctx);
```

Establishes connection with the device and prepares it to perform transactions. The preparation consists of the following sequence:

1. Attempt to disconnect if the connection is already established.
2. Loads configuration from the INI file if it exists.
3. Terminates any "form process" if pending; Initializes the communication object depending on the chosen connection method; Establishes the connection with the device.
4. Performs the device status reset, for a case if the device has a pending command or it is in a transaction mode.
5. Request device version information, validate it and read the device model specific settings from the INI.

6. Synchronizes the device time if the time synchronization is enabled (Enabled by default).
7. Displays the welcome screen on the device.

DeviceConnectRequest Properties

configfile	Should contain the INI file name to be used to configure the library. If it is null or empty then the library will try to locate the INI file in the same directory where of the DLL is located, the names it will try to locate is "[DeviceConnectRequest.DLLclass].ini" (With some variations of the DLLclass) or if DLLclass is null it will try "[Assembly_Name].ini". The use of the INI files is optional.
connectiontype	Connection type to be used to connect to the device. Can be one of: "TCPIP", "Serial", "USB", "Bluetooth" or "AudioJack". Case insensitive. If not specified (Null or empty) then the connection type will be taken from the INI file parameter [Device];ConnectionType. If one of the connection parameters that correspond to the selected connection type is not specified, the parameter value will be taken from the INI file, if it is not present in the INI then the default will be used (Except for the IP address, if IP is missing the connection request will fail).
comport	If the connectiontype is set to "Serial" then this parameter must contain the serial port name. Acceptable range is: "COM1" – "COM255". The default port number is "COM9".
autodetect	The parameter is not used in the current implementation.
baudrate	Baudrate to be used to connect to the serial port. Used only if the connectiontype is set to "Serial", otherwise the value is ignored (Can pass 0 in that case). Supported bitrates are: 9600, 14400, 19200, 38400, 57600, and 115200. The recommended value is 115200. If not specified pass here "-1". The default value is 115200.
databits	Data bits of the serial port. The only supported value at the moment is "8". If the connectiontype is not "Serial" this value is ignored. If not specified pass here "-1". The default value is 8.
parity	Parity settings of the serial port. The acceptable values is: "none", "odd", "even", "mark", "space". Case insensitive. It is recommended to use "none".
stopbits	Stop bits to be used for the serial port configuration. The acceptable values is: "1" or "2". It is recommended to use "1". The parameter is ignored if the connectiontype is not "Serial". If not specified pass here "-1". The default value is 1.
flowcontrol	Serial port handshake type. The acceptable values are: "none", "requesttosend" or "hardware". Case insensitive. It is recommended to use "none".
vendorid, productid	Those parameters specify the Vendor and Product ids associated with a USB device to which we are trying to connect. Those values are used only when the connectiontype is "USB", otherwise it is ignored.
ip	In a case if the connectiontype is "TCPIP" this parameter should contain the network address of the device. The address can be an IP, a domain name or a network MAC address. If it is a domain name and the domain name has

	various IP addresses associated with it (A RECORDs) then each record will be tried consecutively till the connection is established or all records fail. If it is a MAC, then the IP address for the MAC will be taken from the cached network information retrieved with the command “arp -a” (Caution! cleaning the network cash will make the device temporary inaccessible). In a case if the connectiontype is “USB” this parameter should contain the device ID, this ID can be obtained using the device discovery (SLimCDDDevice library), the value should be taken from ICommChannel.Id (Example value: “VP3300:951T397445”). In a case if the connectiontype is “AudioJack” this parameter should contain the device model. Example: known Audio Jack devices: “VP3300”, “UniPay 1.5”, “UniPay”, “UniMagII”, “Shuttle”, “UniMag Pro” (Those values can be hardcoded or obtained through a discovery process).
port	Network port for connecting to the device using “TCPIP” connectiontype, ignored otherwise. If the value of the parameter is 0 – the default port number 12000 will be used by the library (This is the default port configured in Ingenico devices).
ssl	Indicates whether to use SSL encryption for connecting to the device or not. If the parameter is set and true then the encryption settings from the INI file parameters “[TCPIP];Encryption”, “[TCPIP];CertificateName” and “[TCPIP];IgnoreCertErrors” will be used to configure the encryption. If this flag is not set or false then the connection will not be encrypted despite what “[TCPIP];Encryption” says. (Refer to the INI file guide for those setting)
mac_address	The parameter is not used in the implementation.
profile	The parameter is not used in the implementation.
DLLclass	Optional, should contain the name of the device specific assembly (IAssemblyInfo.Name). If provided it is used to locate the device library INI file, if the INI file is not used there is no need to provide this value. CAUTION! When the device libraries are loaded using the legacy SLimCDEMv this parameter is used differently and must contain a specific values. This is a breaking change difference.
vpn_id	The parameter is not used in the implementation.
vpn_username	The parameter is not used in the implementation.
vpn_password	The parameter is not used in the implementation.

Properties Returned in IDeviceInfo

Make	If the connection was established it will contain the device make – “INGNAR” (Ingenico), “IDTech”, “Verifone”, etc.
Model	If the connection was established it will contain the device model – “L3000”, “L5000”, “S4000”, “MX925”, “MX915”, etc.

Disconnecting Device

```
async Task<IResult> DeviceDisconnectAsync(CancellationTokentoken ctx);
```

Performs the device disconnection sequence. First it sends the cancel command to the device to cancel any pending requests and resets the device status, then it displays the “Lane Closed” screen and closes the connection. DeviceConnectAsync needs to be called again to reestablish the connection, any attempts to use the other interface methods beside it will fail. This method will also dispose the internal resources, so if it has been called there is no need for an additional call to Dispose.

Rebooting or Resetting Device

```
Task<IResult> RebootAsync();
Task<IResult> ResetAsync();
```

The function of ResetAsync method is a device and device vendor specific. In a case of Verifone the ResetAsync method will reset the device app without fully rebooting the OS (Faster than a full reboot), in all other cases both methods will do the same thing – fully reboot the device.

After the reboot/reset command is sent to the device the call to those methods internally call DeviceDisconnectAsync to close the connection with the device, so if you are rebooting/resetting you need to reconnect after that.

Configuring Device

```
async Task<IResult> DeviceConfigureAsync(
    string xml,
    Func<string, Task> statusCallback);
```

The method can be used to send a configuration commands to the device. The commands contained in the XML is sent sequentially to the device, if one of the commands fails the process stops and returns the error. The call to this method will return when all commands have finished processing.

Parameters

xml	Have to contain XML with the following structure: <pre><Configuration> <Device> <Commands> <Command action="Send">HEX_BYTES_CMD1</Command> <Command action="Send">HEX_BYTES_CMD2</Command> <Command action="Reboot" /> </Commands> </Device> </Configuration></pre> The listed commands will be sent to the device as a pass-through commands, each command must have a response (After ACK), except the last “Reboot” command. HEX_BYTES_CMD1 must be a continuous string of HEX bytes without any spaces or divisor between bytes.
statusCallback	This callback will be called before and after executing each command with the indication of which command is been called and if it was successful. The before call will look like “Sending command #N”, the after – “Command #N. Success”, where N is the number of the command (Not index). For the reboot only “Rebooting. Success” will be returned. In a case of a failure the messages will be different.

Device Download

```
async Task<IResult> DeviceDownloadAsync(
    string fileName,
    byte[] fileContent,
    Func<string, Task> statusCallback);
```

This method can be used to upload a file to the terminal for example a configuration file, CAPKs, etc.

Parameters

fileName	Required to contain the file name of the full name (Path and the name), this name will be used for saving the file on the device.
fileContent	Optional file content. If it is null then the method will try to load the file content from the storage using the provided fileName as a full name.
statusCallback	Status callback will indicate the uploading progress, and it will be called every time a packet (The size is device specific, can be around 480 Bytes – 8KB) has been uploaded. The text will contain the file name and the upload completion percentage.

Pass-through Commands

```
async Task<IResponse<byte[]>> DevicePassThroughAsync(
    byte[] command,
    bool responseRequired,
    int responseTimeout);
```

Can be used to send any kind of a command to the device. Depending on the settings the command can be with or without response, in a case if the command is with response the method will wait for the device to return the response data and only then will return.

Parameters

command	Contains the data that will be sent to the device.
responseRequired	Indicates the command requires waiting for a response.
responseTimeout	If the response is required this values sets the timeout after which if there is no response the method call will return a timeout failure. If it is <= 0 then there is no timeout and the wait for the response will go indefinitely. If the response is <u>not</u> required (optional) then this value indicates how long we should wait for an optional response, after reaching the timeout the method will return indicating success, but it will not contain any response data. Set this value to 0 for a “fire and forget” commands.

Returns

IResponse<byte[]>.Data	In a case if the call was successful and the response was required the Data will contain the response bytes that was received from the device. If the response was optional the Data will contain the response bytes only if the device returned it during the specified time frame.
------------------------	--

Perform Transaction

Regular transaction:

```
async Task DeviceProcessTransactionAsync(
    DeviceTransactionConfig config,
    DeviceTransactionRequest request,
    Func<DeviceTransactionReply, Task> replyCallback,
    Func<string, Task> statusCallback);
```

Transaction with the IAS callback, "old style":

```
async Task DeviceProcessTransactionAsync(
    DeviceTransactionConfig config,
    DeviceTransactionRequest request,
    Func<DeviceTransactionReply, Task> replyCallback,
    Func<string, Task> statusCallback,
    Func<string, Task<Dictionary<string, string>>> iiasCallback);
```

Transaction with the IAS callback, "new style":

```
async Task DeviceProcessTransactionAsync(
    DeviceTransactionConfig config,
    DeviceTransactionRequest request,
    Func<DeviceTransactionReply, Task> replyCallback,
    Func<string, Task> statusCallback,
    Func<Task<Dictionary<string, string>>> iiasBinLookupCallback);
```

Regular keyed transaction:

```
async Task DeviceProcessKeyedTransactionAsync(
    DeviceKeyedTransactionConfig config,
    DeviceTransactionRequest request,
    Func<DeviceTransactionReply, Task> replyCallback,
    Func<string, Task> statusCallback);
```

Keyed transaction with the IAS callback, "old style":

```
async Task DeviceProcessKeyedTransactionAsync(
    DeviceKeyedTransactionConfig config,
    DeviceTransactionRequest request,
    Func<DeviceTransactionReply, Task> replyCallback,
    Func<string, Task> statusCallback,
    Func<string, Task<Dictionary<string, string>>> iiasCallback);
```

Keyed transaction with the IAS callback, "new style":

```
async Task DeviceProcessKeyedTransactionAsync(
    DeviceKeyedTransactionConfig config,
    DeviceTransactionRequest request,
    Func<DeviceTransactionReply, Task> replyCallback,
    Func<string, Task> statusCallback,
    Func<Task<Dictionary<string, string>>> iiasBinLookupCallback);
```

We have several different methods that allow to perform transactions for a regular and keyed card entry. If you are not planning to use IAS then use the *"Regular transaction"* methods, if you do want to use IAS then there is 2 methods (For each normal and keyed entry): *"old style"* – uses a callback provided by the POS app for the IAS validation (The callback does the validation), *"new style"* – the device library validates the card against the IAS server (SlimCD) and if it is confirmed to be a health card then it call the callback. The old style was used when the card BIN numbers was 6 digits long, which could go in open, unencrypted, but since the BINs length was increased to 8 digits according to the PCI rules we cannot provide the full 8 digits BIN into the POS callback for the merchant to validate it against the IAS therefore the merchant, having an incomplete BIN, cannot reliably determine if the card is a health card or not. Because of that we recommend to use the *"new style"* methods.

Note. The Boolean parameters listed below can be set to "yes", "1" or "true" to enable them or "no", "0" or "false" to disable.

DeviceTransactionConfig Parameters

cfg_tip	Indicates if we should prompt the customer for a tip or not.						
cfg_PINdebit	If set then the “DEBIT” will appear on the card type selection screen (On a Swipe or Keyed).						
cfg_credit	If set then the “CREDIT” will appear on the card type selection screen (On a Swipe or Keyed).						
cfg_gift	If set then the “GIFT CARD” will appear on the card type selection screen. Caution , some of the gift cards contain a non-standard track data (For example “Valutec” cards), to read those cards you need to disable all other card types - cfg_credit, cfg_PINdebit and cfg_loyalty should be set to “0”, otherwise the card read attempt will result in a “Bad card” error.						
cfg_loyalty	If set then the “LOYALTY” will appear on the card type selection screen. Caution , for Ingenico implementation you cannot have both gift and loyalty cards enabled at the same time, it should be one or another.						
cfg_cashback	Indicates the cashback prompt is enabled. The cashback will be prompted to the customer only if the card supports it .						
cfg_surcharge	If this parameter is set and the surcharge amount was not provided in the DeviceTransactionRequest then the library will try to obtain the surcharge amount from the SlimCD server.						
cfg_convenienceFee	When this parameter is set and the convenience fee amount was not provided in the DeviceTransactionRequest then the library will try to obtain this amount from the SlimCD server. Note. Surcharge and convenience fee are mutually exclusive, if the surcharge is set then convenience fee will be disabled.						
cfg_convenienceFeeLabel	Convenience fee name to be used on the device screen and on the receipt.						
cfg_totals	Indicates whether to prompt for the total confirmation or not. In a case of QuickChip the total confirmation also can be managed through the INI configuration.						
cfg_signature	If the transaction requires the signature to be captured and this parameter is set then the customer will be prompted for a signature on the device screen or if it is not set then the signature line will be placed on the receipt. If the device does not support on-screen signature capture then the signature line will also be placed on the receipt regardless.						
cfg_receipt	In a case if the terminal supports receipt printing (For example Ingenico Move/5000) then this parameter can indicate how many receipts to print. It can have the following values: <table border="1"> <tr> <td>0</td><td>Do not print receipts.</td></tr> <tr> <td>1</td><td>Print customer copy.</td></tr> <tr> <td>2</td><td>Print customer and merchant copies.</td></tr> </table>	0	Do not print receipts.	1	Print customer copy.	2	Print customer and merchant copies.
0	Do not print receipts.						
1	Print customer copy.						
2	Print customer and merchant copies.						

cfg_EBT	Indicates the types of the EBT that is supported. If supported then when the EBT card is swiped the terminal will prompt to select the type of the card providing the EBT option to choose. The value is a string that can contain the following options: <table border="1"> <tr> <td>FS</td><td>Food stamps.</td></tr> <tr> <td>CB</td><td>Cash benefits.</td></tr> <tr> <td>CD</td><td>Cash disbursement.</td></tr> </table> The options should be concatenated, for example the value – “FSCB” will enable food stamps and the cash benefits options. Note. In the Ingenico implementation you cannot enable cash benefits and cash disbursement at the same time, must enable one or another.	FS	Food stamps.	CB	Cash benefits.	CD	Cash disbursement.
FS	Food stamps.						
CB	Cash benefits.						
CD	Cash disbursement.						
cfg_FoodStamps	This parameter is not used in the current implementation. See the parameter cfg_EBT to enable food stamps cards.						
cfg_WIC	This parameter is not used in the current implementation. WIC cards is not supported and will be declined.						
cfg_emvEnabled	If the parameter is not set then the device will not allow any contact EMV transaction.						
cfg_swipeEnabled	If the parameter is not set then the device will not allow any swipe transaction.						
cfg_contactlessMode	Indicates the contactless mode. Acceptable values are: “None” – contactless is disabled, “EMV” – contactless is enabled in EMV mode. (There is also “MSR” mode, but it has been deprecated and it is not anymore supported by the brands).						
cfg_quickchip	If set then the device will be working in the QuickChip mode. The transaction flow will be different. The terminal will ask the cardholder to remove the card before sending the transaction for the authorization.						
cfg_deferredAmount	If set then the transaction amount should be provided to the library through the call to DeviceSetDeferredAmount. That allows the customer to insert the card before the final amount is known, the library will read the card and will wait for the amount, then it will process the transaction. Caution! The QuickChip mode will be forced on (cfg_quickchip will be set to “1”) and cashback will be off (cfg_cashback to “0”).						
cfg_emvMerchantDecision	Determines the transaction outcome requested by POS. If it is empty or “0” then we are requesting for the transaction to be approved (TC), “1” – requesting for the transaction to go online, any other value will mean requesting a decline. It is recommended to use “0” or just leave it empty. Important. Please consult our support if this feature is supported by our L3 certification with the processor that you are using.						
cfg_emvConnectivityStatus	This parameter is not used in the current implementation.						
cfg_swipeReceiptThreshold	This parameter is not used in the current implementation.						

cfg_NOCVM_SignatureThreshold	This parameter can be set to an amount below which the terminal will not be prompting the cardholder to enter the signature on the device screen, then if the signature is needed the receipt will contain the signature line. Example: "50.00".
cfg_requireEncryption	If set indicates that the device encryption must be enabled. If the device does not have keys or has the disabled encryption all transaction attempts will fail. Must be enabled in production.

The keyed transaction has a majority of the parameters that match the parameters located in DeviceTransactionConfig, and has the same meaning with the exception of listed below.

DeviceKeyedTransactionConfig Parameters

cfg_cvv2prompt	Indicates if to prompt for CVV entry or not.
cfg_zipprompt	Indicates if to prompt for ZIP code or not.
cfg_addressprompt	This parameter is not used in the current implementation. The address entry is not supported.

DeviceTransactionRequest Parameters

username, password, clientid, siteid, priceid	Those are the credentials provided by SlimCD. If instead of getting all those parameters you got an API Access Credential, place it in username, then instead of the password you put string.Empty, and "0" - for clientid, siteid and priceId.						
product	Name of the POS software. This will be shown on SlimCD client site. Used for convenience.						
version	Version of your POS software. This will be shown on SlimCD client site. The library will add "E" on the end of the version to identify it as "EMV" version of the library.						
key	Credentials for the server authentication, received from SlimCD. This credential is optional and it's not been used in the most cases.						
transtype	The transaction types is supported by the library: <table border="1"> <tr> <td>Sale</td><td>A regular sale/purchase, creates the transaction that will be settled in the next batch.</td></tr> <tr> <td>Auth</td><td>An authorization of funds that will not be settled till the Force transaction is called. Force should be called with gateid received in the Auth response.</td></tr> <tr> <td>Credit</td><td>Performs a refund to the customer account. Specify the gateid of the transaction that is been refunded or if there is no such, then you need to add the parameter blind_credit set to "1" into industry_specifics dictionary to enable this behavior, and also the blind credit should be enabled in the merchant setting.</td></tr> </table>	Sale	A regular sale/purchase, creates the transaction that will be settled in the next batch.	Auth	An authorization of funds that will not be settled till the Force transaction is called. Force should be called with gateid received in the Auth response.	Credit	Performs a refund to the customer account. Specify the gateid of the transaction that is been refunded or if there is no such, then you need to add the parameter blind_credit set to "1" into industry_specifics dictionary to enable this behavior, and also the blind credit should be enabled in the merchant setting.
Sale	A regular sale/purchase, creates the transaction that will be settled in the next batch.						
Auth	An authorization of funds that will not be settled till the Force transaction is called. Force should be called with gateid received in the Auth response.						
Credit	Performs a refund to the customer account. Specify the gateid of the transaction that is been refunded or if there is no such, then you need to add the parameter blind_credit set to "1" into industry_specifics dictionary to enable this behavior, and also the blind credit should be enabled in the merchant setting.						

	<table border="1" data-bbox="646 142 1414 254"> <tr> <td data-bbox="646 142 820 254">Load</td><td data-bbox="820 142 1414 254">Stores the card information and returns a token "gateid" that can be used to initiate other transactions without requiring card entry.</td></tr> </table> Transactions with other types can also be sent using this library, however they do not require card entry and the library will just pass it through to the server. Instead you can use the direct requests to the SlimCD server to send it. Please refer to our "<i>Slim CD Transaction Gateway Manual</i>" for the details on the other transaction types. Note. In case of a Void transaction when you need the card to be entered it is possible to force the card entry by specifying the parameter <code>void_need_card_entry</code> equal to "1" in <code>industry_specifics</code> of the Void transaction request. Otherwise the card entry is not performed and the Void request is just passed through to the server.	Load	Stores the card information and returns a token "gateid" that can be used to initiate other transactions without requiring card entry.
Load	Stores the card information and returns a token "gateid" that can be used to initiate other transactions without requiring card entry.		
authcode	Auth code received for Offline transactions (Voice referral transactions, the code is given by the bank on the phone).		
clienttransref	Client-supplied transaction identifier, (i.e. invoice) which will be passed to the credit card network for specific industry types. Max size is 20 characters.		
gateid	Should contain the transaction id of the transaction that have to be voided, forced, refunded, etc. This id should be obtained from the approval transaction response. In a case of a Void If this gateid is 0 the gateway will try to void the last transaction received. However, it is highly recommended to always pass the transaction id for voids.		
amount	Amount of the transaction, should be in a range from 0.01 to 999999.99. The value must contain digits and the decimals dot divisor, only 2 decimals is allowed. In a case of a deferred amount transaction this amount will be ignored. Format "N.NN" where 'N' is a number. The value can be 0.00 for Auth transactions for the processors that support it.		
gratuity	Tip amount, the value should be in a range from 0.01 to 999999.99. The amount must have this tip amount already added to it. If the tip flag is set in DeviceTransactionConfig then the customer will also be prompted for the additional tip that will be added to the total.		
surcharge	Surcharge amount, the value should be in a range from 0.01 to 999999.99. The amount must have this surcharge amount already added to it. If this surcharge amount is present and greater than 0 and <code>cfg_surcharge</code> is also set then <code>cfg_surcharge</code> will be ignored and no request to obtain the surcharge from the server will be made, the value from this parameter will be used instead.		

allow_partial	Indicates if the partial approvals should be accepted. If not set then the transactions that could be partially approved will be declined. The acceptable values only “yes” or “no”.
allow_duplicates	If set it disables the transaction duplication control. The duplicate identification works by the card number, amount and the client transaction reference in the past 30 minutes. If the transaction is considered duplicate the duplicate attempt is declined. It is highly recommended to keep the duplicate control on in production. The acceptable values only “yes” or “no”.
emvmode	Keep this parameter empty, it is used internally.
postype	In a case if POS is an Electronic Cash Register must contain the value – “ECR”.
firstname	Additional transaction data that will be passed to the gateway.
lastname	Additional transaction data that will be passed to the gateway.
address	Additional transaction data that will be passed to the gateway.
city	Additional transaction data that will be passed to the gateway.
state	Additional transaction data that will be passed to the gateway.
zip	Additional transaction data that will be passed to the gateway.
country	Additional transaction data that will be passed to the gateway.
phone	Additional transaction data that will be passed to the gateway.
email	Additional transaction data that will be passed to the gateway.
corporatecard	Corporate card indicator. The acceptable values only “yes” or “no”.
po	Purchase order number. Optional, maximum 20 characters.
salestax	Amount of sales tax. Example: “5.00”.
salestaxtype	Sales tax type of the amount provided in salestax. Acceptable values is: 0 – N/A, 1 – Tax Amount, 2 – Exempt Amount.
clerkname	Pass here the clerk name or id of the clerk who has initiated the transaction.
deviceid	An id that preferably uniquely identifies the POS. Used for convenience, displayed on the SlimCD client site.
industry_specifics	Dictionary of a key-value pairs containing the parameters that has to be passed to the gateway.

Other Parameters

replyCallback	Upon transaction completion this callback will be invoked providing the transaction completion data (See below the detailed parameters description). The device library invoked this
---------------	--

	callback asynchronously, so it can be called after the process transaction method has returned.
statusCallback	This callback will be called during the transaction process to provide the current status of the transaction to POS. The messages will be in a short readable form, apt to be displayed on POS.
iiasCallback	The “old style” IIAS callback. It will be called with the 6 digits of the card BIN and then it should determine if it is a health card, if it is then it should return to the library a set of parameters detailing the amount of the transaction. In particular: 1. Set the “IIASIndicator” to “yes”. 2. Set the “IIASVerified” flag to “yes” if the individual items were verified, “no” if they were not. 3. Set the “Amount” field to the total amount of the sale. 4. Set the “AmtHealth4S” field to the amount to be charged as Health Benefits. And then specify the AmtHealth4S details: 5. Set the “AmtPrescr4U” field to the total amount of the RX or Prescription products. 6. Set the “AmtVision4V” field to the amount of the Vision/Optical products/services. 7. Set the “AmtDental4X” field to the amount of the Dental products/services. 8. Set the “AmtClinic4W” field to the amount of the Clinic or Other qualified medical products/services.
iiasBinLookupCallback	The new style IIAS callback, before calling this callback the library will make a call to the SlimCD server IIAS service to determine if the card is the health card, in this call it will send the complete encrypted card data so the server can access the full card number to determine if it is a health card or not, then if it is a health card the library will invoke this callback. The server validation of the card against IIAS is needed because only the first 6 digits of a BIN is accessible at the POS level, the server having the complete card number can match it to the 8 digit BINs or other.

DeviceTransactionReply Response Parameters

response	Will equal “Success” in a case if the transaction was approved, “Error” if the transaction received decline from the server, or “Fail” if the transaction was declined locally as for example declined by chip. Should be case insensitive.
responsecode	Will be equal 0 in a case if the transaction was approved, “2” if the transaction received decline from the server, or “1” if the transaction was declined locally as for example declined by chip.
description	Will contain the description of the transaction result – a completion message or an error description.
responseurl	URL that have processed request and returned this response.

approved	Will contain "Y" or "B" if the transaction was approved. Note that the following logic have to be implemented: IF approved = Y or approved = B THEN approvedamt is the amount actually approved IF amount - approvedamt > 0 THEN Collect any remaining balance END IF END IF								
authcode	Processor transaction authorization code.								
cvv2reply	The parameter is not used in the current implementation.								
avsreply	The parameter is not used in the current implementation.								
gateid	Transaction ID generated by SlimCD server.								
bankid	Will contain the bank Id received from the processor.								
corporatecardindicator	Indicates if it is a corporate card. The types can be: <table border="1"> <tr> <td>"B" or "O"</td><td>Business card.</td></tr> <tr> <td>"R"</td><td>Corporate card.</td></tr> <tr> <td>"S"</td><td>Purchasing card.</td></tr> <tr> <td>"L"</td><td>B2B settlement card.</td></tr> </table>	"B" or "O"	Business card.	"R"	Corporate card.	"S"	Purchasing card.	"L"	B2B settlement card.
"B" or "O"	Business card.								
"R"	Corporate card.								
"S"	Purchasing card.								
"L"	B2B settlement card.								
invoiceno	Invoice/Folio Number received from the processor.								
firstname	First name on the card.								
lastname	Last name on the card.								
cardtype	Will contain the card brand, such as "American Express", "Visa", "MasterCard", "Discover" or "PIN Debit" in a case of "DNA".								
first6	The 6 digits of the card BIN number.								
last4	The last 4 digits of the card number.								
expmonth	Card expiration month number.								
expyear	The last two digits of the card expiration year.								
processor_token	Token returned from the processor.								
approvedhsamt	The portion of the approved amount that corresponds to the health card spending.								
approvedamt	The total amount that was approved.								
bal	Balance returned by the issuer (Gift card balance or other).								
keyed	Indicates if the transaction was keyed or not. Can be "yes" or "no".								
pindebit	Indicates if the card is debit and the PIN was entered.								
tip	Amount that was charged as tip.								
cashback	Amount that was charged as cashback.								

surcharge	Amount that was charged as surcharge.
entrymode	Indicates how the card data was entered to the device. Can be one of: "Chip Read", "Swiped", "Keyed", "Contactless Chip", "Contactless MSR", "Fallback Swiped" or "Fallback Keyed".
signaturestatus	The status of the signature capture. Can be one of "NotNeeded", "Needed" (Signature line should be printed on the receipt and signed by the customer) or "Captured".
receiptstatus	The status of the receipt, can be "NotNeeded", "Needed" or "Printed". In the current implementation this parameter has a fixed value "Needed".

Set the Deferred Amount

```
async Task DeviceSetDeferredAmountAsync(
    DeviceDeferredAmountRequest deferredAmount);
```

The deferred amount transactions can be initiated before the final transaction amount is known (By setting the parameter `cfg_deferredAmount`), the device will display a prompt for a card entry without showing the amount, the card is entered and the card data is read, then the device stops and indicates to the customer to wait and waits for a call to this method from the POS. When the POS knows the final amount it calls this method and the transaction continues till completion. In a case of a regular EMV the card must remain the entire time inside of the terminal, however in a case of QuickChip the card can be extracted before the final amount is known.

The deferred amount transaction is not compatible with the contactless EMV mode, if the contactless is enabled the deferred mode will switch it off. This is due to the fact that for contactless the final amount must be known at the moment on the transaction initiation as it is "a part" of the cryptogram. However in the latest device certification Contactless QuickChip mode has been added, allowing to perform contactless transactions while using the deferred mode. Contact out support for more information.

DeviceDeferredAmountRequest Parameters

amount	Indicates the final amount. Must be in a range from 0.01 to 999999.99.
gratuity	The tip amount. Must be already included in the amount.
surcharge	The surcharge amount. Must be already included in the amount.
conveniencefee	The convenience fee amount. Must be already included in the amount.
conveniencefee_label	The convenience fee label to be shown on the device screen and on the receipt.
doTip	If true then the device will prompt for the tip entry.
doSurcharge	If the surcharge is not set or equals 0 and this flag is true then the library will request the surcharge amount from the SlimCD server. If surcharge has a value greater than 0 this flag will be ignored.
doConvenienceFee	If the conveniencefee is not set or equals 0 and this flag is true then the library will request the convenience fee amount from the

	SlimCD server. If conveniencefee has a value greater than 0 this flag will be ignored.
--	--

Note. Surcharge and convenience fee are mutually exclusive, if one is set the other must be disabled, if both are set at the same time then the convenience fee will be switched off.

Cancelling Transaction

```
async Task<DeviceTransactionCancelReply> DeviceTransactionCancelAsync();
```

Calling this method will cancel a transaction that is currently in process. At the moment it is not allowed to cancel the transaction after it has been submitted to the server, if this method is invoked at that moment it will return an error indicating that the cancellation is not possible.

To cancel the transaction the library checks if the command that is currently been executed is waiting for a customer input, if it does then it sends the status reset commands to the device and waits for a response data to arrive, the time to wait may vary from – about 300 milliseconds to 3 seconds or never, depending on the command and other factors, because of that, the cancellation wait time can be substantial - up to 3-4 seconds and it cannot be reduced without risking errors with the next transaction.

The method also has a timeout of 5 seconds, if the cancellation fails to complete in in this time it will return an error.

If the transaction was successfully cancelled the response will indicate that the transaction was cancelled by the attendant.

DeviceTransactionCancelReply Response Parameters

response	Will be equal "Success" if the transaction was successfully cancelled or "Error" if the cancellation have failed.
responsecode	Will be equal "0" if the transaction was successfully cancelled or "1" if the cancellation have failed.
description	Will contain the description of the transaction result – a completion message or the error description.

Skipping Signature Entry

```
bool DeviceIsWaitingForEntry()
```

A call to this method will return True if the device is currently waiting for the customer to enter the signature or False if there is no waiting entry (Or if the device library doesn't support checking the entry status). If the return value is True then you can call DeviceSkipEntry to skip the signature entry.

```
IResult DeviceSkipEntry()
```

This method when called will attempt to skip the signature entry if the device is currently waiting for one. If successful the method will return right away, it will not wait for the transaction results to be sent to the callback.

IResult Response Parameters

IsSuccess	When set it means that the signature entry will be skipped and the library will call the transaction results callback in a few moments.
-----------	---

IsFailure	Mean that we are unable to skip the signature entry, that can be because there are currently no signature entry waiting or that the device library doesn't support this functionality. If the transaction results callback was not called yet you may attempt to cancel the transaction or dispose of the Hardware object to reset.
Message	In a case if IsFailure is True it will contain the description why we were not able to skip the entry.

Forms Management Process

The forms management allows to display, populate and update forms (screens) on the device. To begin to work with forms we call `DeviceProcessFormStart` to start the forms **session**. When the session is started the library is reconfigured internally, `DeviceTransactionCancel` will not work and no API requests should be done directly, only `DeviceProcessFormLineItem`, `DeviceProcessFormUpdate` or `DeviceProcessFormEnd` should be called. Starting a transaction will internally close the forms session without performing `DeviceProcessFormEnd`.

The forms session will not be able to start if there is a transaction in the process.

Starting the Forms Session

```
async Task DeviceProcessFormStartAsync(
    DeviceFormRequest request,
    Func<IResponse<DeviceFormReplyBase>, Task> responseCallback);
```

Initiates a forms session, **loads the form** specified in the request and sets the parameters. The response callback provided in this method will be used to return a response from calls to `ProcessFormUpdate` and `ProcessFormLineItem`, although those methods does have the callbacks that is provided to them, those callbacks will be called only if there is an immediate failure, the callback provided here will be called to return other responses. We recommend calling the method `ProcessFormStart` only once and then use the "update" methods to do the rest for an optimal experience, due to that `ProcessFormStart` loads the form and that usually adds a significant overhead (For example calling `ProcessFormStart` on Verifone will take 740ms and performing an update using `ProcessFormUpdate` only 220ms).

Note. In a case of Verifone a single batch command is used to display and update the form.

DeviceFormRequest Parameters

FormName	Name of the form to be displayed and updated.
Inputs	The parameter is not used in the current implementation.
Prompts	Contains the text that has to be assigned to the controls with the specified ids. First the text is cleared or set (if provided), then the control visibility is triggered according to the settings.
Buttons	Contains the buttons information, new button text can be provided to replace the existing and the buttons visibility settings.
CheckBoxes	Contains the settings for the checkboxes, sets or clears the checkmark and then applies the control visibility settings.
RadioButtons	Contains the settings for the radio buttons, sets or clears the selection and then applies the control visibility settings.

TextBoxes	Contains the text that has to be assigned to the text boxes. If the text is provided it is set or if it is indicated it is cleared, then the control visibility is triggered according to the settings.
-----------	---

Other Parameters

responseCallback	Callback that will be called to return a response. Depending on the implementation, the response call may be omitted.
------------------	---

Ending Forms Session

```
async Task DeviceProcessFormEndAsync();
```

Terminates the forms session – reconfigures the library to work in the normal mode and **clears the screen**.

Updating the Forms

```
async Task DeviceProcessFormUpdateAsync(DeviceFormRequest request);
```

Updates the form during the forms session, the object structure for the information to be updated is the same as it is described in chapter on DeviceProcessFormStart. A single batch command is used for the update in a case of Verifone.

Adding a Line Item

```
async Task DeviceProcessFormLineItemAsync(DeviceFormLineItemRequest request);
```

If specified by the flag – clears the list boxes and then adds the list box items. The clearing and updating of the form is done using a single batch command.

DeviceFormLineItemRequest Parameters

ControlId	Id of the list box control that have to be updated.
ClearDisplay	Indicates if to clear the list before adding the items or not.
LineItems	An array of the items that need to be inserted to the list box. Each item is represented with the object of the type FormLineItem that contain the parameters Text – the text to be inserted and LineNumber – the line number of where it have to be inserted. If LineNumber is "0" then the item will be inserted at the beginning of the list, if it is "-1" then the item will be added in the end of the list.
LineItemTotals	The parameter is not used in the current implementation.

Other Parameters

callback	Callback that will be called only in a case of an error otherwise the method will return without calling it. If the form is supposed to provide a response, the response will be returned in a callback provided in ProcessFormStart.
----------	---

Interactive API

The `ISlimCDDDeviceAsync` instance on the device API may also implement `IInteractive` interface (Defined in `SLimCDTypeLib`). This interface consists of a several events that can be raised by the device library to request user input on the POS app. This functionality is optional and intended to be used with the readers that does not poses a screen or a required input capability. To use it just subscribe to the event, the library will detect the subscription and will raise the event when needed, if those events has no subscribers then the library will be using a default behavior which would depend on a case.

IInteractive events:

MessageEventHandler StatusUpdate	Will be called for all device library status updates. Those updates should be shown to the cardholder and contain what is usually shown on the reader screen.
TotalsUpdateEventHandler TotalsUpdate	Will be called to update the transaction totals when the transaction amounts has changed (tip, surcharge, etc.)
UserSelectEventHandler UserSelect	Called when the device library requests the user to select an option. (Not used at the moment)
UserConfirmEventHandler UserConfirm;	Called when the device library requests the user to confirm something.
UserSignatureEventHandler UserSignature;	Called when the device library requests the user to enter signature.
UserEntryEventHandler UserEntry;	Called when the device library needs for the user to enter data. (Not used at the moment)

ICommManager API

The `ISlimCDDDeviceAsync` instance on the device API may also implement `ICommManager` interface (Defined in `SLimCDTypeLib`). This API allows you to discover and monitor devices as well as to obtain the supported connection types and more. The data and the functionality of this API is platform specific and can be redefined by providing `IPlatformSpecific` at the moment of creating the API instance (See `SLimCDDDevice` guide).

Get the Supported Connection Types

```
IReadOnlyCollection<ConnectionType> SupportedConnectionTypes { get; set; }
```

Specifies a collection of the connection types supported by the device library on the current platform. Can be one of, but not limited: Serial, TcpIp, Usb, Bluetooth, AudioJack.

Device Connection Monitoring

```
bool DeviceConnected { get; }
event DeviceConnectedEventHandler DeviceConnectedChanged;
```

The `DeviceConnected` property will indicate if the device is currently connected. The `DeviceConnectedChanged` event will be raised in a case if the connection status has changed, the event argument will indicate the current connection status.

Get Communication Capabilities

```
async Task<CommCapabilities> GetCommCapabilities(ConnectionType connectionType);
```

The call will return the capabilities flags that is specific to the device library, the current platform and the provided connection type. Get the capabilities to determine if a certain comm features are available before using them.

CommCapabilities flags:

Discover	Indicates that the communication channels discovery is supported. (Like discovery of the Serial Ports, USB, Bluetooth devices).
Monitor	Indicates that the communication channels monitoring is supported.
Create	Indicates that the device specific library implements its own communication manager instead of using the default provided in DeviceFramework.
ConnectionChanged	Indicates the DeviceConnectedChanged event is supported, otherwise the event will not be raised and DeviceConnected will be always False.

Discovering Communication Channels

```
async Task<IResponse<IReadOnlyCollection<ICommChannel>>> DiscoverComm(
    ConnectionType connectionType,
    int timeout,
    CancellationToken ctx);
```

Performs communication channels discovery for the specified connection type. The returned collection may contain the list of Serial Ports, USB devices, paired Bluetooth devices, etc. Each returned communication channel can be used to attempt to establish a connection with the device.

Parameters

connectionType	Connection type that should be used to perform devices discovery.
timeout	Timeout, when reached the method will return right away. If no devices were discovered during this time an empty collection will be returned.
ctx	Cancellation token, causes the method to return right away.

Returns

IReadOnlyCollection<ICommChannel>	Returns a collection a devices discovered using the specified connection type or null if the discovery is supported, but not implemented indicating that the default implementation should be used to perform the discovery. If the discovery is not supported in CommCapabilities an error will be returned. If the timeout has been reached or cancellation triggered then it will return an empty list. If cancelled, an error will be returned.
-----------------------------------	---

Monitoring Communication Channels

```
async Task<IResponse<IDisposable>> MonitorComm(  
    ConnectionType connectionType,  
    ObservableCollection<ICommChannel> channels,  
    Func<Action, Task> invoker);
```

Starts monitoring devices for a specified connection type. The devices are added/removed to/from the provided ObservableCollection.

Parameters

connectionType	Connection type for which the monitoring should be performed.
channels	The collection where the current list of the channels will be maintained.
invoker	Invoker that will be used to perform operations on the devices collection. Use it in a case if it is required to perform the collection operations on the main UI thread.

Returns

IDisposable	Returns the devices monitoring object that is IDisposable. It will return an error if CommCapabilities indicates that Monitoring is not supported. May return null if CommCapabilities indicates that the monitoring is supported, but it is not implemented, that indicates that default comm monitoring should be used. When the returned object is disposed it will stop the monitoring.
-------------	---

Note. The other public method that is exposed by this interface is CreateComm, this method is for internal use and should not be called directly.